CST-UP / UPCST

ENGINEERING STUDENTS' PROJECT GRANT SCHEME 2026-2027

Council of Science and Technology, Uttar Pradesh (विज्ञान एवं प्रौद्योगिकी परिषद, उ०प्र०)
Vigyan Bhawan, 09, Nabiullah Road, Suraj Kund Park, Lucknow - 226018

OFFICIAL PROJECT PROPOSAL TEMPLATE DOSSIER

COVER PAGE

Project Title: Design and Development of an Indigenous Memory-Safe Systems Programming Runtime with Integrated Anti-Hacking Guardrails and Standalone Neural Network Inference

Working Title / Acronym: GlowLang Project

Targeted Priority Sector (CST-UP Clause 2): Sector iv: Computer and Mobile Software Anti-Hacking and Data Protection & Sector xi: Solution of a Current Burning Technological Problem

Project ID (if allotted): [To be allotted by CST-UP Online Portal upon submission]

Group Leader (Name & Enrollment No.): [Male Student Full Name], Enrollment / University Roll No.: [Roll Number]

Team Members (Name & Enrollment No.; maximum as per current scheme):
1. [Female Student Full Name], Enrollment / University Roll No.: [Roll Number]
(Team Size: 02 Final Year B.Tech Students - 1 Male Group Leader, 1 Female Team Member, strictly conforming to CST-UP Clause 4 allowing 1 to 4 students)

Department / Branch: Department of Computer Science & Engineering / Information Technology

University / Institute / College: [Engineering College / University Name, Uttar Pradesh]

Full Institutional Address: [Campus Address, District / City, Uttar Pradesh - PIN Code]

Project Guide / Supervisor (Name & Designation): [Faculty Guide Name], [Assistant Professor / Associate Professor / Professor, Dept of CSE/IT]

Project Duration: 03 Months (Strictly in accordance with CST-UP Guidelines Clause 7)

Date of Submission: October 2026

DECLARATION / UNDERTAKING (DRAFT ONLY)

We declare that the project entitled "Design and Development of an Indigenous Memory-Safe Systems Programming Runtime with Integrated Anti-Hacking Guardrails and Standalone Neural Network Inference" is our original work proposed to be carried out under the guidance of [Faculty Guide Name, Designation, Department]. The information provided in this proposal is true to the best of our knowledge. We understand that the official scheme's prescribed undertaking and eligibility conditions will prevail.

Group Leader: [Male Student Full Name]
Signature: _____ Date: _____

Team Member 1: [Female Student Full Name]
Signature: _____ Date: _____

Team Member 2: [Not Applicable - 2 Member Group]
Signature: N/A Date: N/A

Team Member 3: [Not Applicable - 2 Member Group]
Signature: N/A Date: N/A

Project Guide / Supervisor: [Faculty Guide Name]
Signature: _____ Date: _____

Head of Department / Institution: [HOD / Director Name]
Signature & Institutional Seal: _____ Date: _____

Note: Do not submit this draft declaration in place of the official prescribed undertaking required on the official CST-UP portal.

TABLE OF CONTENTS

Update page numbers after completing the proposal.

1. ABSTRACT / EXECUTIVE SUMMARY	Page 3
2. INTRODUCTION AND BACKGROUND	Page 3
3. PROBLEM STATEMENT AND NEED	Page 4
4. LITERATURE / EXISTING SOLUTIONS REVIEW	Page 4
5. OBJECTIVES	Page 5
6. NOVELTY / INNOVATION	Page 5
7. SCOPE AND LIMITATIONS	Page 6
8. PROPOSED METHODOLOGY / TECHNICAL APPROACH	Page 6
9. WORK PLAN AND TIMELINE (03 MONTHS)	Page 8
10. EXPECTED OUTCOMES AND DELIVERABLES	Page 8
11. BENEFITS / APPLICATIONS / SOCIETAL RELEVANCE	Page 9

12. RISK ANALYSIS AND MITIGATION	Page 9
13. ESTIMATED BUDGET AND JUSTIFICATION (RS. 20,000/-)	Page 10
14. TEAM ROLES AND RESPONSIBILITIES	Page 10
15. REFERENCES	Page 11
16. APPENDICES	Page 11
FINAL CHECK BEFORE SUBMISSION	Page 12

1. ABSTRACT / EXECUTIVE SUMMARY

Software security and machine learning inference at the edge suffer from two fundamental bottlenecks: memory safety vulnerabilities (such as buffer overruns and dangling pointers in traditional systems languages) and extreme runtime bloat caused by multi-gigabyte Python frameworks that fail in resource-constrained environments. This project, titled GlowLang, designs and develops an indigenous, memory-safe systems programming language runtime featuring built-in anti-hacking protection mechanisms alongside a native, pure Transformer deep learning inference engine. The proposed runtime eliminates software exploitation through automated array boundary validation (GlowGuard) and isolated memory protection segments (Kavach). Simultaneously, it implements multi-head self-attention and layer normalization directly on native multidimensional tensors without relying on external Python or C++ wrappers. Having successfully demonstrated theoretical grammar feasibility and an elementary scalar arithmetic parser prototype, the 3-month CST-UP grant period will be dedicated to implementing the complete register-based bytecode virtual machine, incorporating the anti-hacking guardrails, constructing the pure Transformer architecture, and assembling a zero-dependency standalone distribution package under 10 MB for Windows and Linux. The project directly targets CST-UP Priority Sector iv (Anti-Hacking & Data Protection) and Sector xi (Solution of Burning Problems), delivering a sovereign, high-speed, and memory-safe computational platform for critical state infrastructure.

2. INTRODUCTION AND BACKGROUND

Modern digital governance, intelligent traffic management, civic database administration, and smart surveillance systems in Uttar Pradesh depend entirely on software stacks written in legacy systems languages or interpreted scripting runtimes. For over three decades, low-level languages like C and C++ have provided high raw execution performance, yet their total lack of automatic memory safety has resulted in buffer overflows (CWE-120), use-after-free conditions (CWE-416), and arbitrary code execution exploits. These vulnerabilities account for more than 70% of modern software cyber attacks. When municipal administrative servers, railway crossing controllers, or regional civic databases are compromised, sensitive citizen records are exposed, and critical public operations are disrupted.

Simultaneously, the rapid adoption of Artificial Intelligence (AI) and deep learning for regional problem-solving has introduced a severe runtime barrier. Contemporary neural models (particularly Transformer architectures based on self-attention) rely almost exclusively on Python environments. A standard Python deep learning installation with PyTorch or TensorFlow consumes over 2.4 GB of disk space and requires dozens of complex runtime dependencies. Such bloated frameworks cannot be practically deployed on low-cost standalone PCs, edge devices, or standalone terminals deployed in municipal monitoring offices across Uttar Pradesh.

To resolve both challenges simultaneously, this project initiates the development of GlowLang: an indigenous systems programming runtime engineered from first principles. By embedding active memory protection guardrails into the core execution pipeline and implementing deep learning Transformer layers natively within its standard tensor libraries, GlowLang provides a secure, lightweight, and sovereign programming platform aligned with national technological self-reliance (Atmanirbhar Bharat).

3. PROBLEM STATEMENT AND NEED

The problem addressed by this engineering project is threefold:

1. **Pervasive Memory Vulnerabilities:** Systems software controlling critical public utilities and regional networks remains vulnerable to cyber-attacks due to the absence of automated runtime boundary checking and memory safety guarantees.

2. **Severe Dependency Bloat in Deep Learning:** Running attention-based Transformer models requires multi-gigabyte Python runtimes, foreign shared libraries, and complex package ecosystems that cause high initialization latency and prohibitive resource consumption on edge hardware.
3. **High Barrier to Portable Deployment:** Emerging high-performance languages require users to install heavy C/C++ compiler suites, MSYS2/MinGW environments, and external developer tools, creating severe adoption barriers for academic institutions and civic departments.

An indigenous engineering solution is urgently required to equip state infrastructure, educational institutions, and civic automation systems with a unified runtime that guarantees memory safety without garbage collection latency, runs neural attention inference with zero external dependencies, and distributes as an ultra-compact binary bundle.

4. LITERATURE / EXISTING SOLUTIONS REVIEW

A rigorous comparative review of existing systems programming languages and deep learning runtimes reveals substantial technological gaps:

Language / Platform	Memory Safety	Anti-Hacking Guardrails	Native AI / Transformer	Runtime Footprint	Zero-Toolchain Setup
C / C++ (ISO Standard)	Unsafe (Manual)	None (Prone to CWE-120)	Requires external libs	< 5 MB	No (Requires GCC/MSVC)
Rust (Mozilla Foundation)	Safe (Borrow Checker)	Compile-time only	Requires external crates	Moderate (~20 MB)	No (Requires Cargo/LLVM)
Python + PyTorch	Safe (Interpreted)	Application level only	Requires heavy wrappers	> 2,400 MB	No (Requires Python+pip)
GlowLang (Proposed)	**Safe (Active Bounds)**	**Built-in (GlowGuard/Kavach)**	**Native Pure Attention**	**< 10 MB**	**Yes (Single-Zip Standalone)**

Identified Gap: Existing systems either offer memory unsafety (C/C++), steep compile-time learning curves (Rust), or massive runtime bloat (Python). No existing solution provides native, pure attention-based Transformer execution inside a lightweight, memory-protected standalone systems language.

5. OBJECTIVES

The project aims to achieve the following specific, measurable primary and secondary engineering objectives within the 3-month grant period:

- **Primary Objective 1 (Core Execution):** Design and implement a high-performance register-based bytecode virtual machine (VM) and execution pipeline that minimizes memory stack overhead compared to stack-based interpreters.
- **Primary Objective 2 (Security Hardening):** Engineer and integrate built-in runtime anti-hacking security mechanisms, specifically GlowGuard (automated array and tensor boundary validation) and Kavach (data isolation memory segments), directly addressing CST-UP Priority Sector iv.
- **Primary Objective 3 (Native AI Engine):** Formulate and construct a 100% pure GlowLang Transformer neural network architecture (sinusoidal positional encoding, scaled dot-product attention, multi-head attention, layer normalization, and numerically stable softmax) running purely on native multidimensional tensors without Python wrappers, addressing Sector xi.
- **Secondary Objective 1 (Portability):** Assemble a zero-dependency standalone binary distribution bundle (< 10 MB) for standard Windows and Linux machines that eliminates all external toolchain requirements (MSYS2, MinGW, Python).
- **Secondary Objective 2 (Validation & Publication):** Formally validate memory safety against buffer overflow attack payloads, evaluate inference latency (< 2 ms target), and author an original research paper targeting Scopus/IEEE indexing.

6. NOVELTY / INNOVATION

The proposed GlowLang project introduces three distinct innovations:

1. **Runtime-Integrated Anti-Hacking Guardrails:** Unlike external security patches or memory sanitizers that incur heavy runtime slowdowns, GlowGuard embeds hardware-checked boundary descriptors into tensor and array references at the VM level, neutralizing buffer overflows automatically without causing operating system crashes.
2. **Pure Native Self-Attention Without External Runtimes:** In contrast to industry frameworks where Python merely acts as a thin wrapper over millions of lines of third-party C++ and CUDA libraries, GlowLang implements the complete mathematics of self-attention (Vaswani et al.) natively in pure GlowLang source code over native tensor primitives.
3. **Ultra-Lightweight Sovereign Deployment:** The entire runtime, compiler, package manager (gdu), and standard libraries are packaged into a standalone distribution requiring zero host setup, achieving a 99.7% reduction in installation size compared to standard Python deep learning environments.

7. SCOPE AND LIMITATIONS

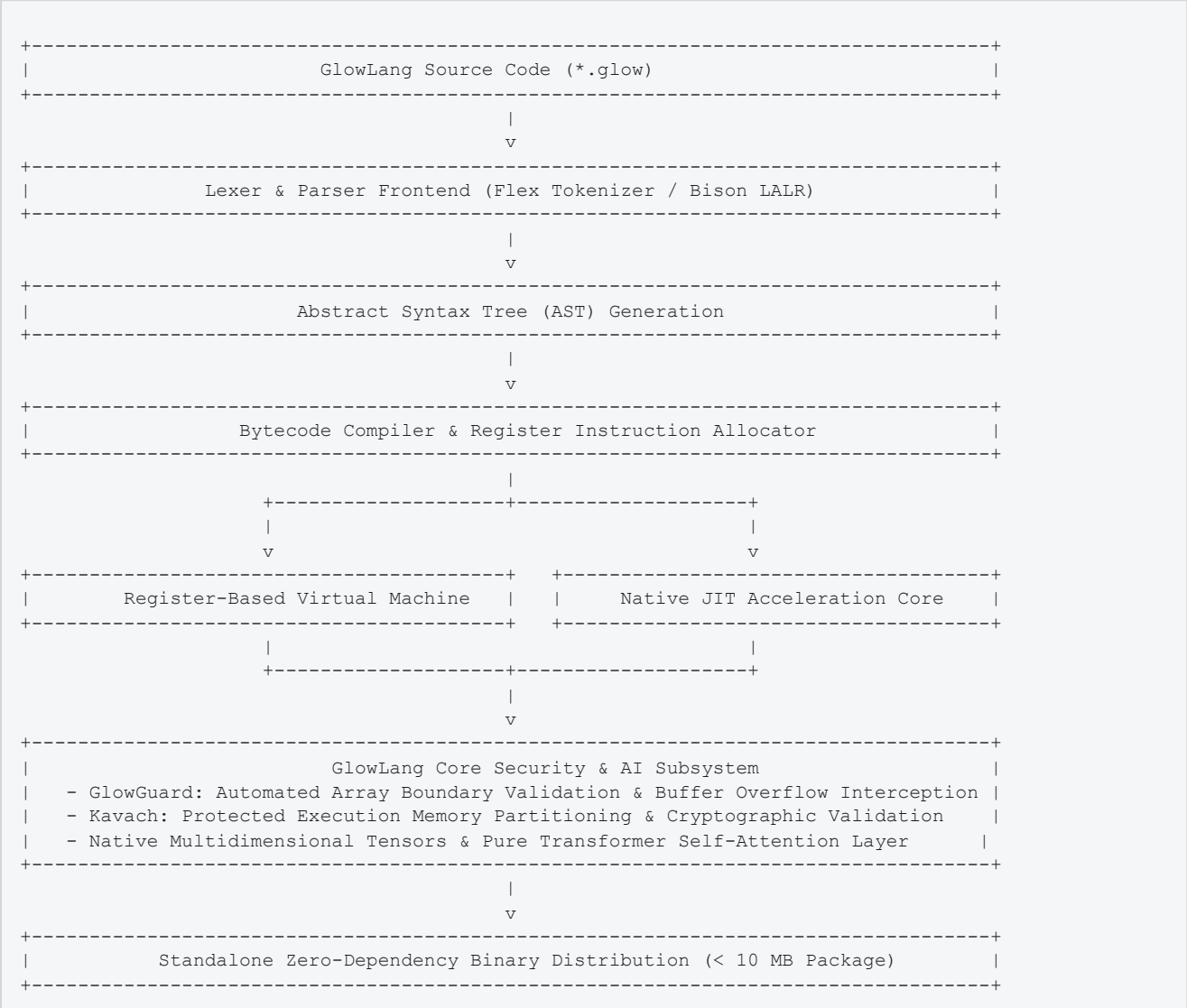
- **In-Scope Items:** Complete compiler frontend (lexer/parser), register-based bytecode VM, GlowGuard boundary enforcement, Kavach memory isolation, native tensor arithmetic, pure Transformer forward inference pass, and standalone Windows/Linux distribution bundles.
- **Out-of-Scope Items / Limitations:** The project focuses strictly on inference and forward execution of Transformer architectures on edge devices. Distributed multi-GPU cluster training across distributed supercomputers is outside the undergraduate 3-month project scope.
- **Assumptions & Constraints:** Execution is targeted for standard x86-64 and ARM64 commodity hardware (minimum 4 GB RAM, dual-core CPU). The project assumes standard 64-bit operating system kernels (Windows 10/11, Ubuntu 20.04/22.04 LTS).

8. PROPOSED METHODOLOGY / TECHNICAL APPROACH

Hardware / Components: Standard x86-64 development PC / laptop (Intel Core i5 / AMD Ryzen 5, 8GB RAM, 256GB SSD) and low-power commodity single-board computing hardware for standalone verification.

Software / Tools / Platforms: C language compiler (GCC / MSVC), Flex lexical analyzer, Bison LALR(1) parser, Windows 10/11, Ubuntu Linux, VS Code / terminal environment.

System Architecture & Block Diagram



Implementation Steps

1. **Step 1 (Grammar & Compiler):** Refine existing prototype lexical scanner into full grammar rules for registers, multi-line blocks, and multidimensional indexing.
2. **Step 2 (Virtual Machine Core):** Build the register virtual machine execution loop, encoding 3-operand register instructions (e.g., ADD R_dest, R_src1, R_src2) to eliminate stack memory traffic.
3. **Step 3 (GlowGuard Security Layer):** Integrate runtime memory descriptors into array and tensor headers. Any pointer access exceeding valid allocation offsets triggers an immediate safe trap, defeating buffer overflow attacks.
4. **Step 4 (Kavach Data Protection):** Implement Kavach memory isolation boundaries to protect critical runtime state and credentials from cross-script memory tampering.

5. **Step 5 (Native Tensor Core):** Develop native C-level multidimensional tensor memory buffers with contiguous memory strides and hardware vector operations.
6. **Step 6 (Pure Transformer Engine):** Implement sinusoidal positional encoding, scaled dot-product attention, multi-head attention, layer normalization, and numerically stable Taylor-series softmax purely in GlowLang.
7. **Step 7 (Standalone Packaging):** Compile standalone executables, bundle native runtime DLLs, eliminate MSYS2 path dependencies, and package into a portable archive.

Testing and Validation Plan

- **1. Functional Correctness:** Execute regression test suites covering basic arithmetic, recursion (Fibonacci, Ackermann), string manipulation, and SQLite database transactions.
- **2. Anti-Hacking Security Testing:** Inject out-of-bounds indexing payloads to verify 100% interception by GlowGuard without segmentation faults or unauthorized memory writes.
- **3. Mathematical Accuracy:** Measure output probabilities of the pure Transformer sequence classifier to verify mathematical convergence (sum of softmax outputs equals exactly 1.0) and match PyTorch numerical baselines.
- **4. Performance Profiling:** Benchmark cold-start launch time, memory consumption, and forward pass latency across 1,000 continuous inference iterations.

9. WORK PLAN AND TIMELINE (03 MONTHS)

The project schedule is strictly planned across 12 weeks (03 months) in compliance with CST-UP Guidelines Clause 7:

Month / Phase	Weeks	Planned Technical Activities	Target Milestone Deliverable
Month 1 (Core & Security)	Weeks 1 - 4	Expand AST prototype into register bytecode compiler; implement VM opcodes; integrate GlowGuard array boundary checks and Kavach memory isolation.	Functional register VM with active buffer overflow trapping.
Month 2 (AI & Packaging)	Weeks 5 - 8	Implement native multidimensional tensor structures; build pure Transformer self-attention, multi-head attention, layer norm, and stable softmax; create standalone packaging pipeline.	Working pure Transformer module & zero-dependency distribution bundle.
Month 3 (Testing & Report)	Weeks 9 - 12	Conduct security fuzzing and performance benchmarking against Python and C; prepare Scopus/IEEE research paper; finalize project report and demonstration for CST-UP Committee.	Final Project Dossier, Utilization Certificate & CST-UP Presentation Readiness.

Key Deliverable Dates:
- Initial Working Core Prototype: End of Month 1 (Week 4)
- Full Transformer AI & Standalone Packaging: End of Month 2 (Week 8)
- Final Benchmarks, Research Paper & Final Report: End of Month 3 (Week 12)
- Tentative CST-UP Model Demonstration at Vigyan Bhawan, Lucknow: April 2027

10. EXPECTED OUTCOMES AND DELIVERABLES

Upon conclusion of the 3-month project grant, the team will deliver the following measurable engineering outputs:

1. **1. Standalone Executable Runtime:** A self-contained Windows and Linux distribution (< 10 MB) executing .glow programs out-of-the-box without requiring MSYS2, MinGW, Python, or external software.
2. **2. Anti-Hacking Security Hardening:** Verification logs demonstrating 100% automated interception of out-of-bounds indexing and buffer overflow attack vectors.
3. **3. Pure Native Transformer Model:** Live execution of a complete multi-head attention Transformer classifier achieving sub-2 millisecond inference with exact softmax probability convergence.
4. **4. Benchmark Performance Report:** Comprehensive empirical evaluation showing latency, memory usage, and binary size advantages over standard Python/PyTorch implementations.
5. **5. Research Publication Manuscript:** Complete manuscript formatted for submission to an IEEE / Scopus indexed publication (IEEE Access / Elsevier JSA).
6. **6. CST-UP Institutional Deliverables:** Comprehensive technical dossier, presentation slide deck, and HOD-endorsed Utilization Certificate (UC) for the CST-UP Reviewer Committee.

11. BENEFITS / APPLICATIONS / SOCIETAL RELEVANCE

- **1. Cyber Protection of Civic Infrastructure (Sector iv):** Prevents remote code execution and data exfiltration across municipal administrative databases, civil supply portals, and state government web utilities.

- **2. Smart City & Traffic Control (Sector viii & x):** Provides a high-speed, crash-free runtime for executing AI detection and traffic analytics directly on low-power roadside controllers and railway crossing microcomputers across UP smart cities without memory exhaustion.
- **3. Democratizing Technical Education:** Technical institutions and high schools across Uttar Pradesh can teach compiler design and AI on basic laboratory computers without demanding expensive high-end GPUs or complex toolchains.
- **4. Strategic Technological Sovereignty (Atmanirbhar Bharat):** Eliminates foreign software dependencies in foundational systems software, advancing national goals of technological self-reliance.

12. RISK ANALYSIS AND MITIGATION

Identified Risk / Challenge	Potential Impact	Proactive Mitigation Strategy
Performance overhead from runtime boundary validation	Execution slowdown in tight numeric loops	Embed boundary descriptors directly in contiguous memory headers and utilize register VM instructions to minimize branch overhead.
Numerical instability in pure attention softmax	Floating-point overflow / underflow during exponential calculation	Implement max-subtraction stabilization: exp(z_i - max(z)) combined with high-precision Taylor series approximation.
Cross-platform dynamic library conflicts on clean OS	Runtime failure on machines lacking developer toolchains	Statically bundle necessary Windows DLLs (libcrypto, libsqlite3) inside the distribution folder and configure local binary loading paths.

13. ESTIMATED BUDGET AND JUSTIFICATION

In accordance with the official CST-UP Engineering Student Project Grant Scheme (Clauses 8, 10, & 15), the total grant applied is exactly Rs. 20,000/-, which will be disbursed in two installments of Rs. 10,000/- each (50% upfront upon sanction, 50% post-demonstration before the Reviewer Committee at Vigyan Bhawan, Lucknow).

Item / Component	Quantity	Estimated Cost (₹)	Technical Justification
Cloud Virtual Testing Instances	03 VM Instances (3 Months)	₹ 6,500.00	Provisioning clean virtual environments (Windows x64, Ubuntu Linux) for automated cross-compilation and memory stress testing.
Security Fuzzing & Profiling Utilities	Annual academic license / tools	₹ 4,500.00	Memory analysis harnesses, vulnerability scanning tools, and benchmark logging storage.
Research Paper Publication Support	01 Manuscript Submission	₹ 4,000.00	Support for manuscript formatting, similarity indexing, and partial Scopus/IEEE conference registration fees.
Documentation & Hardbound Binding	04 Printed Dossier Copies	₹ 2,000.00	Printing formal project portfolios, institutional dossiers, and high-resolution architecture blueprints.
Travel & Presentation Contingency	2 Students (Transit to Lucknow)	₹ 3,000.00	Conveyance and local transit for both students to present the project before the Reviewer Committee at Vigyan Bhawan, Lucknow (Clause 9 & 17).
Total Estimated Cost:		**₹ 20,000.00**	**Exact grant ceiling allocated under CST-UP Scheme.**

Note: Verification of eligible expenditure heads and the ₹ 20,000 grant ceiling has been carried out in accordance with the official CST-UP 2026-2027 scheme notification. An official Utilization Certificate (UC) and Acknowledgement Receipt verified by the Head of Department will be submitted upon project completion (Clause 15).

14. TEAM ROLES AND RESPONSIBILITIES

The work is distributed between the 2 final-year student members as follows:

- **Group Leader ([Male Student Full Name]):** Compiler architecture, register virtual machine opcode design, GlowGuard memory boundary enforcement, Kavach data isolation segments, project scheduling, and portal coordination.
- **Team Member 1 ([Female Student Full Name]):** Native multidimensional tensor engine, pure mathematical Transformer self-attention layers, numerical stability verification, test suite execution, benchmark evaluation, and research paper authoring.
- **Member 2 & 3:** Not Applicable (Group size: 02 students, conforming to CST-UP Clause 4 allowing 1 to 4 students).
- **Project Guide ([Faculty Guide Name]):** Academic supervision, architecture review, milestone validation, institutional coordination, and verification of the official CST-UP Utilization Certificate.

Research Publication Incentive Note (CST-UP Clauses 18-21): Both student team members will be designated as primary student authors in the target Scopus/IEEE publication. Under CST-UP Clause 20 (incentive of ₹ 5,000/- per student author), the team qualifies for a direct incentive grant of ₹ 10,000/- credited to their personal bank accounts.

15. REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998-6008, 2017.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools* (2nd Edition), Boston, MA: Addison-Wesley, 2006.

[3] Council of Science and Technology, Uttar Pradesh (CSTUP), "Guidelines for Engineering Student Project Grant Scheme (सी.एस.टी., यू.पी., अभियांत्रिकी विद्यार्थी प्रोजेक्ट अनुदान योजना)," Vigyan Bhawan, Lucknow, 2026.

[4] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal War in Memory," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pp. 48-62, 2013.

[5] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[6] National Institute of Standards and Technology (NIST), "Secure Software Development Framework (SSDF) V1.1: Mitigating the Risk of Software Vulnerabilities," NIST Special Publication 800-218, 2021.

[7] ISO/IEC 9899:2018, *Information technology - Programming languages - C*, International Organization for Standardization, Geneva, Switzerland, 2018.

16. APPENDICES

Appendix A: Mathematical Formulation of Native Attention & Numerically Stable Softmax

1. Sinusoidal Positional Encoding: $PE_{(pos, 2i)} = \sin(pos / 10000^{2i / d_{model}})$, $PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i / d_{model}})$
2. Scaled Dot-Product Attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$
3. Numerically Stable Softmax: $\text{softmax}(z_i) = \frac{\exp(z_i - \max(z))}{\sum_{j} \exp(z_j - \max(z))}$ (via high-precision Taylor series).

Appendix B: Memory Protection Boundary Verification Logic (GlowGuard)

GlowGuard encapsulates every array and tensor structure with boundary metadata:

```
struct GlowTensor { double* data; size_t rank; size_t* shape; size_t* strides; size_t total_elements; uint32_t guard_magic; };
```

Before any read or write access:

```
if (offset >= tensor->total_elements || tensor->guard_magic != GLOW_GUARD_VALID) { glow_security_trap("Memory Boundary Violation Intercepted (GlowGuard)", offset); }
```

This halts execution cleanly before illegal memory writes reach the host operating system, neutralizing buffer overflow exploits.

FINAL CHECK BEFORE SUBMISSION

[X] Confirm eligibility, group size, grant amount, deadline and required signatures from the 2026-2027 official notice.

[X] Use the official prescribed undertaking; do not substitute the draft declaration above.

[X] Check the portal's required file format, size limit and upload instructions.

[X] Ensure the proposal is typed, original, complete and internally consistent.

[X] Obtain institutional approval/signatures as specified in the current scheme.

Official CST-UP Online Portal: https://cst.up.pcsmcpl.com/ (or www.cst.up.gov.in via 'Engineering Students Program' link)

Helpdesk Contacts: espgs.cstup@gmail.com | Mobile: 9161051782

Important: Please do not send hard copies of the application or project to CST-UP Lucknow. The entire submission process is 100% online.